

Part I: Fundamentals of Deep Learning

RTG Summer School

Yimin Zhong (yimin.zhong@auburn.edu)

Probability & Concentration

Definition (Probability Space)

A triple $(\Omega, \mathcal{F}, \mathbb{P})$ where:

1. Ω is the **sample space**
2. $\mathcal{F}$ is a σ -algebra (collection of events)
3. $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$ is a probability measure

random variable $X : \Omega \rightarrow E$ is a measurable function. Its expectation:

$$\mathbb{E}[X] = \int_{\Omega} X(\omega) d\mathbb{P}(\omega)$$

variance: $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.

Jensen's inequality: For convex function h , $\mathbb{E}[h(X)] \geq h(\mathbb{E}[X])$.

Types of Convergence

Almost surely: $\mathbb{P}(\lim X_n = X) = 1$

In L^p : $\mathbb{E}|X_n - X|^p \rightarrow 0$

In probability: $\forall \varepsilon > 0 : \mathbb{P}(|X_n - X| > \varepsilon) \rightarrow 0$

In distribution: $\mathbb{E}[f(X_n)] \rightarrow \mathbb{E}[f(X)]$ for all bounded continuous f

Implications:

a.s. $\Rightarrow$ prob. $\Rightarrow$ dist. ; $L^p \Rightarrow$ prob. $\Rightarrow$ dist.

Law of Large Numbers

Theorem (Weak Law of Large Numbers)

Let $X_1, \dots, X_n$ be i.i.d. with $\mathbb{E}[X] = \mu$. Then $Z_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{p} \mu$.

Proof: $\mathbb{P}(|Z_n - \mu| > \varepsilon) \leq \frac{\text{Var}(Z_n)}{\varepsilon^2} = \frac{\sigma^2}{n\varepsilon^2} \rightarrow 0.$ (Chebyshev)

Theorem (Strong Law of Large Numbers)

Let $X_1, \dots, X_n$ be i.i.d. with $\mathbb{E}[X] = \mu$. Then a.s. $Z_n \rightarrow \mu$.

Central Limit Theorem

Theorem (Central Limit Theorem)

With $\sigma^2 = \text{Var}(X)$:

$$\frac{\sqrt{n}(Z_n - \mu)}{\sigma} \xrightarrow{d} \mathcal{N}(0, 1)$$

A large number of **small, independent** random effects sum to approximately a Gaussian.

Theorem (Berry-Esseen)

With $\sigma^2 = \text{Var}(X)$ and $\rho = \mathbb{E}[|X|^3]$, let F_n be the empirical CDF of $\frac{\sqrt{n}}{\sigma}(Z_n - \mu)$, then

$$\sup_{x \in \mathbb{R}} |F_n(x) - \Phi(x)| \leq \frac{C\rho}{\sigma^3 \sqrt{n}}.$$

This shows $O(n^{-1/2})$ uniform control over the entire distribution from a Gaussian.

Theorem (Bayes)

$\mathbb{P}(w \mid \mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|w)\mathbb{P}(w)}{\mathbb{P}(\mathcal{D})}$, *that is, posterior $\propto$ likelihood $\times$ prior.*

- **MLE:** Maximize $\mathbb{P}(\mathcal{D} \mid w)$, frequentist, data only
- **MAP:** Maximize $\mathbb{P}(w \mid \mathcal{D})$, Bayesian, includes prior $\rightarrow$ regularization
- L_2 regularization = Gaussian prior on w .

Exercise: Berry-Esseen Sharpness

Exercise 1

The Berry-Esseen theorem states that for i.i.d. variables with finite third moment, the CLT convergence rate is $O(n^{-1/2})$. Use Bernoulli(1/2) random variables to show this rate is sharp.

Hint: The Bernoulli distribution has point masses. What happens to the CDF of $\sqrt{n}(Z_n - \mu)/\sigma$ at points between the atoms?

Concentration Inequalities: Chernoff Bound

For sharper tails with **independence**, we use concentration:

Theorem (Chernoff Bound — Multiplicative)

Let $X = \sum_{i=1}^n X_i$ where X_i are independent Bernoulli(p_i), $\mu = \mathbb{E}[X]$. For any $\varepsilon > 0$:

$$\mathbb{P}[X \geq (1 + \varepsilon)\mu] \leq \exp\left(-\frac{\varepsilon^2}{2 + \varepsilon} \mu\right)$$

For $\varepsilon \in (0, 1)$:

$$\mathbb{P}[X \leq (1 - \varepsilon)\mu] \leq \exp\left(-\frac{\varepsilon^2}{2} \mu\right)$$

Combined two-sided: $\mathbb{P}[|X - \mu| \geq \varepsilon\mu] \leq 2 \exp(-\varepsilon^2\mu/3)$.

Exponential decay in $\mu = \sum p_i$.

Hoeffding's Inequality

Theorem (Hoeffding)

Let X_i be independent with $a_i \leq X_i \leq b_i$. Set $X = \sum_i X_i$, $\mu = \mathbb{E}[X]$. Then:

$$\mathbb{P}[|X - \mu| \geq t] \leq 2 \exp\left(-\frac{2t^2}{\sum_i (b_i - a_i)^2}\right)$$

Comparison with Chernoff (assume i.i.d. Bernoulli p):

$$\text{Hoeffding: } 2 \exp(-2\varepsilon^2 np^2), \quad \text{Chernoff: } 2 \exp(-\varepsilon^2 np/3)$$

- For small p : Chernoff is **much** sharper (linear in p vs. quadratic)
- For large p : comparable

Concentration inequalities quantify how many samples are required for reliable estimates.

Numerical Comparison

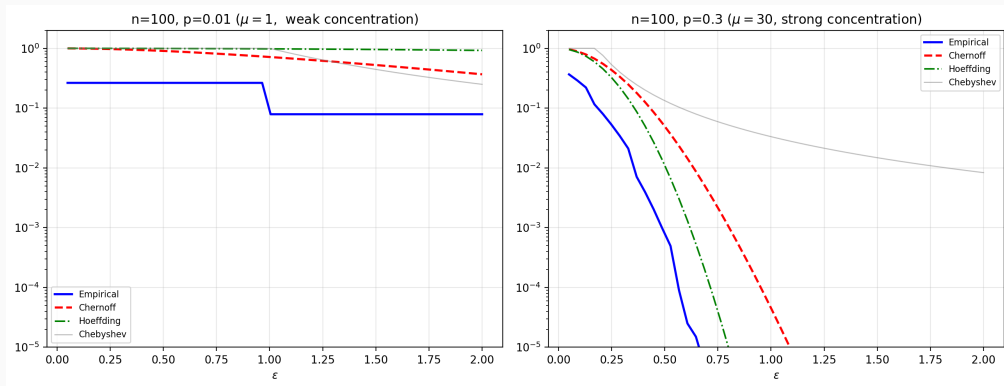


Figure 1: MC vs. Chernoff vs. Hoeffding vs. Chebyshev

Monte Carlo: Using Concentration to Control Error

Goal: Estimate $\mu = \mathbb{E}[f(X)]$ where $X \sim P$, using n i.i.d. samples.

Monte Carlo estimator: $\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n f(X_i)$.

Error bounds:

- **Chebyshev:** $|\hat{\mu}_n - \mu| \leq \sigma / \sqrt{n\delta}$ with probability $\geq 1 - \delta$
- **Hoeffding** (if $f \in [a, b]$): $|\hat{\mu}_n - \mu| \leq (b - a) \sqrt{\frac{\log(2/\delta)}{2n}}$ with prob. $\geq 1 - \delta$

Example (Estimating π)

Throw n random points in $[0, 1]^2$, count the fraction inside the unit circle centered at the origin.

$$\hat{\pi}_n = 4 \cdot \frac{\#\{\text{in circle}\}}{n}, \quad \text{error} \sim O(1/\sqrt{n})$$

The $O(1/\sqrt{n})$ rate is universal for Monte Carlo.

Monte Carlo: Numerical Demonstration

Estimating $\pi = 3.14159 \dots$ by throwing n random points into $[0, 1]^2$:

$$\hat{\pi}_n = 4 \cdot \frac{\#\{\text{in unit circle}\}}{n}$$

n	$\hat{\pi}_n$	$ \hat{\pi}_n - \pi $	$1/\sqrt{n}$
10^2	3.12	0.022	0.100
10^3	3.144	0.002	0.032
10^4	3.1412	0.0004	0.010
10^5	3.14144	0.00015	0.0032
10^6	3.141548	0.00004	0.0010

Error scales as $\approx 1/\sqrt{n}$ (CLT), not converging fast but easy to parallelize.

Hoeffding bound (for Bernoulli with $p = \pi/4$): $|\hat{\pi}_n - \pi| \leq \sqrt{\frac{2 \log(2/\delta)}{n}}$ with probability $\geq 1 - \delta$.

For $\delta = 0.05$, $n = 10^6$: error $\leq \sqrt{2 \log(40)/10^6} \approx 0.0027$.

- **Empirical risk minimization:** The empirical loss $\hat{R}_n(w) = \frac{1}{n} \sum_i \ell(w, x_i)$ concentrates around the true risk $R(w) = \mathbb{E}_X[\ell(w, X)]$. Hoeffding gives $|\hat{R}_n - R| \leq O(\sqrt{\frac{\log(2/\delta)}{2n}})$ with probability $\geq 1 - \delta$.
- **Minibatch SGD:** A minibatch of size B estimates the full gradient with variance $O(\sigma^2/B)$. Concentration controls the noise level.

Information Theory

Shannon Entropy: Measuring Uncertainty

Definition (Shannon Entropy)

For a discrete random variable X with distribution $p(x) = \mathbb{P}[X = x]$:

$$\mathbb{H}(X) = \mathbb{E}[-\log p(X)] = - \sum_x p(x) \log p(x)$$

Intuition: How many bits (**on average**) are needed to encode the outcome of X ?

- Fair coin: $\mathbb{H} = -2 \cdot \frac{1}{2} \log_2 \frac{1}{2} = 1$ bit
- Biased coin ($p = 0.99$): $\mathbb{H} \approx 0.08$ bits
- Uniform over n outcomes: $\mathbb{H} = \log n$ bits

Thermodynamic connection: $S = -k_B \sum p_i \log p_i$ (Boltzmann entropy). The form is identical: entropy measures disorder in both physics and information theory.

From Concentration to Typical Sequences

Flip a biased coin ($p = 0.9$) n times. How many **typical** sequences are there?

Step 1 — Concentration: By Hoeffding, the number of heads H concentrates:

$$\mathbb{P}(|H - 0.9n| > \varepsilon n) \leq 2 \exp(-2\varepsilon^2 n) \xrightarrow{n \rightarrow \infty} 0$$

With probability $\rightarrow 1$, $H \in [0.9n - O(\sqrt{n}), 0.9n + O(\sqrt{n})]$.

Step 2 — Count typical sequences: The number of sequences with exactly k heads is $\binom{n}{k}$. For $k \approx pn$, **Stirling's approximation** gives:

$$\binom{n}{pn} \approx \frac{1}{\sqrt{2\pi np(1-p)}} \cdot 2^{nH(p)}, \quad H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$$

The dominant term is $2^{nH(p)}$ — exponential in n , with exponent $H(p)$.

Step 3 — The entropy emerges: For $p = 0.9$, $H(0.9) \approx 0.469$. A sequence of $n = 100$ flips has $\binom{100}{90} \approx 2^{47}$ typical outcomes with ≈ 90 heads. Only these require codes; the rest are vanishingly rare on average as $n \rightarrow \infty$.

Entropy = Bits

Biased coin, $p = 0.9, n = 100$ flips:

1. Concentration: $H \approx 90 \pm 3$ (with $\varepsilon = 0.03$, probability $\geq 1 - 2e^{-0.18} \approx 83\%$)
2. Number of sequences with $H \in [87, 93]$:

$$\binom{100}{90} \approx \frac{1}{\sqrt{2\pi \cdot 100 \cdot 0.09}} \cdot 2^{100 \cdot 0.469} \approx \frac{1}{7.5} \cdot 2^{46.9}$$

3. Bits needed: $\log_2(\frac{1}{7.5} \cdot 2^{46.9}) \approx 44$ bits for 100 flips $\rightarrow$ **0.44 bits/flip**.
4. As $n \rightarrow \infty$: the bits per flip $\rightarrow H(0.9) = 0.469$ exactly.

Fair coin, $p = 0.5, n = 100$:

$$\binom{100}{50} \approx \frac{1}{\sqrt{50\pi}} \cdot 2^{100 \cdot 1.0} \approx 2^{100} \text{ sequences} \rightarrow 100 \text{ bits} \rightarrow \mathbf{1 \text{ bit/flip}}$$

Definition (Conditional Entropy)

$$\mathbb{H}(X | Y) = - \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(y)} = \mathbb{H}(X, Y) - \mathbb{H}(Y)$$

The uncertainty about X that remains after observing Y .

Chain rule: $\mathbb{H}(X, Y) = \mathbb{H}(X | Y) + \mathbb{H}(Y) = \mathbb{H}(Y | X) + \mathbb{H}(X)$. Knowing X and Y jointly = knowing Y , then X given Y (or vice versa).

Subadditivity: $\mathbb{H}(X, Y) = \mathbb{H}(X) + \mathbb{H}(Y | X) \leq \mathbb{H}(X) + \mathbb{H}(Y)$ since $\mathbb{H}(Y | X) \geq 0$. Equality iff $X \perp\!\!\!\perp Y$. Extends to n variables: $\mathbb{H}(X_1, \dots, X_n) \leq \sum_i \mathbb{H}(X_i)$.

Definition (Mutual Information)

$$I(X; Y) = \mathbb{H}(X) - \mathbb{H}(X | Y)$$

*The reduction in uncertainty about X after observing Y . Equivalently, $I(X; Y) = \mathbb{H}(Y) - \mathbb{H}(Y | X)$.
Mutual information is symmetric. $I(X; Y) = 0$ iff X and Y are independent.*

Entropy of Deterministic Functions

Key identity: $\mathbb{H}(f(X) \mid X) = 0$. Once X is known, $f(X)$ has zero remaining uncertainty.

Therefore, using the chain rule:

$$\mathbb{H}(X) = \mathbb{H}(f(X)) + \mathbb{H}(X \mid f(X)) \geq \mathbb{H}(f(X))$$

Processing never increases entropy. Any deterministic function can only destroy or preserve information, never create it.

Question

How to explain that a pseudorandom number generator (PRNG) can generate random numbers?

Submodularity of Entropy

For any random variables X, Y, Z :

$$\mathbb{H}(X, Z) + \mathbb{H}(X, Y) \geq \mathbb{H}(X) + \mathbb{H}(X, Y, Z)$$

More generally, for index sets I and J :

$$\mathbb{H}(\{X_i\}_{i \in I}) + \mathbb{H}(\{X_i\}_{i \in J}) \geq \mathbb{H}(\{X_i\}_{i \in I \cap J}) + \mathbb{H}(\{X_i\}_{i \in I \cup J})$$

Interpretation: Entropy exhibits **diminishing returns**, i.e., a variable added to a small set yields more new information than the same variable added to a large set.

Submodularity in Action

Exercise 2 (modified from this post)

Let X_1, X_2, X_3, X_4 be independent **positive** random variables. Define:

$$Z = (X_1 + X_2)(X_3 + X_4),$$

$$W = (X_1, X_2, X_3, X_4),$$

$$U = (X_1 + X_2, X_3 + X_4),$$

$$V = (X_1X_3, X_1X_4, X_2X_3).$$

1. Prove $H(Z) + H(W) \leq H(U) + H(V)$. Show Z is a function of U ; Z is a function of V too. W is a function of (U, V) .

2. Now assume $X_1, \dots, X_4$ are i.i.d. copies of X , and let Y be an independent copy. Derive:

$$H(Z) + 4H(X) \leq 2H(X + Y) + 3H(XY)$$

$H(W) = 4H(X)$, $H(U) = 2H(X + Y)$ by independence; $H(V) \leq 3H(XY)$ by subadditivity.

Application: The 12-Ball Puzzle

Problem: 12 balls, one of which weighs differently from the others (heavier or lighter). Find it in minimal weighings with a balance scale.



Application: The 12-Ball Puzzle

Problem: 12 balls, one of which weighs differently from the others (heavier or lighter). Find it in minimal weighings with a balance scale.



Information Perspective:

- Total uncertainty: 12 balls $\times$ 2 states = 24 possibilities $\rightarrow \mathbb{H} = \log_2 24 \approx 4.58$ bits.
- Each weighing has 3 outcomes $\rightarrow$ at most $\log_2 3 \approx 1.58$ bits of **mutual information**.
- Three weighings: at most $3 \times 1.58 = 4.75$ bits ≥ 4.58 bits — **possible!**
- This bound is a theoretical lower bound.

Exercise: The Number-Guessing Game

Exercise 3

A secret code is 3 **distinct** digits from $\{0, 1, 2, 3, 4\}$ in order (e.g., “3-1-0”). Each guess receives one answer: how many digits are completely correct (correct digit at correct position). Possible answers: 0, 1, 2, or 3. How many guesses are necessary?

Exercise 4

Implement an automated solver for “Wordle”.

Definition (Kullback-Leibler Divergence)

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

The “cost” of approximating the true distribution P with the model Q .

Theorem (Gibbs Inequality)

$D_{KL}(P||Q) \geq 0$, equality iff $P = Q$.

Not symmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$ in general. $D_{KL}(P||Q)$ is large when $Q(x)$ is small where $P(x)$ is large (model underestimates likely events). $D_{KL}(Q||P)$ penalizes overestimation.

Question

Why is P usually the data distribution and Q is the model prediction distribution?

Cross-Entropy and Maximum Likelihood

Cross-entropy (CE):

$$H(P, Q) = - \sum_x P(x) \log Q(x) = \mathbb{H}(P) + D_{KL}(P||Q)$$

When P is fixed (data), minimizing $H(P, Q) \iff$ minimizing $D_{KL}(P||Q)$.

With the empirical data distribution $\hat{P}(x) = \frac{1}{n} \sum_{i=1}^n \delta_{x_i}(x)$:

$$H(\hat{P}, Q) = -\frac{1}{n} \sum_{i=1}^n \log Q(x_i)$$

This is the **negative log-likelihood**. Minimizing the cross-entropy loss is equivalent to maximum likelihood estimation. *This is why every deep learning classification model minimizes cross-entropy.*

Exercise: KL Properties

Exercise 5

Taking Q as the uniform distribution over a finite set Ω , prove that $\mathbb{H}(P) \leq \log |\Omega|$ for any distribution P on Ω . *Hint: $D_{KL}(P||Q) \geq 0$.*

Exercise 6

Let $P = \text{Uniform}([-1, 1])$ and $Q = \text{Uniform}([-2, 2])$. Compute $D_{KL}(P||Q)$. Does your answer make sense?

Language Modeling: Cross-Entropy Loss

A language model essentially estimates $Q(w_t \mid w_{<t})$ to each token given context w .

Cross-entropy loss (per token):

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log Q(w_t \mid w_{<t})$$

This is exactly the cross-entropy $H(\hat{P}, Q)$ between the empirical next-token distribution $\hat{P}$ and the model Q . **It provides an upper bound of the entropy of $\hat{P}$.**

Perplexity: $\text{PPL} = \exp(\mathcal{L})$ estimates how predictable a language is (on average, how many choices for next-token)

Entropy of English: Landmark Estimates

How predictable is natural language? Key estimates of bits per character:

- **Shannon (1951)**: Human letter-guessing $\rightarrow$ 0.6 to 1.3 bits/char.
- **Cover & King (1978)**: Gambling-based estimate $\sim$ 1.25 bits/char.
- **Brown et al. (1992)**: Trigram on Brown Corpus $\rightarrow$ 1.75 bits/char (7.95 bits/word, PPL $\approx$ 247).

Modern neural models reach far lower: GPT-2 $\approx$ 1.0–1.2 bits/char; Llama-3 pushes below 0.8 bits/char, approaching the lower bound for human-level prediction.

Perplexity Scaling in Practice

Example (WikiText-103 (approximate, from GPT-2, Llama-3 papers))

<i>Model</i>	<i>Parameters</i>	<i>Perplexity</i>
<i>LSTM</i>	<i>0.2B</i>	<i>~ 50</i>
<i>GPT-2</i>	<i>1.5B</i>	<i>~ 18</i>
<i>Llama-3</i>	<i>70B</i>	<i>~ 5</i>

Each generation of the new model reduces perplexity by capturing more linguistic structure.

Exercise: Estimating Language PPL

Try to compute the PPL for Brown et al.'s (1992) data.

Exercise 6

Build bigram $n = 2$ or trigram $n = 3$ models: $Q(w_t \mid w_{t-1}, \dots, w_{t-n+1})$ and compute cross-entropy and PPL on test data.

Q1: What does the unigram $n = 1$ model compute?

Q2: What is the complexity of building the n -gram model?

Network Architectures

The Perceptron

The simplest neural unit: a linear classifier with a threshold.

$$f(x) = \text{sgn}(w \cdot x + b)$$

Identifies a separating hyperplane $w \cdot x + b = 0$.

Perceptron algorithm (Rosenblatt, 1958): For each misclassified point (x_i, y_i) , update $w \leftarrow w + y_i x_i$. Converges in finite steps if data is linearly separable.

The XOR problem (Minsky & Papert, 1969): A single perceptron cannot separate XOR — the points $\{(0, 0), (1, 1)\}$ vs $\{(0, 1), (1, 0)\}$ are not linearly separable. This result killed early neural network research (first AI winter).

Multilayer Perceptron (MLP)

Adding **hidden layers** resolves XOR and enables universal approximation:

$$h^{(1)} = \sigma(W^{(1)}x + b^{(1)}), \quad h^{(2)} = \sigma(W^{(2)}h^{(1)} + b^{(2)}), \quad f(x) = W^{(3)}h^{(2)} + b^{(3)}$$

Architecture: L hidden layers, width k_l per layer. Fully connected: every neuron in layer l connects to every neuron in layer $l + 1$.

Activation functions:

- **Sigmoid:** $\sigma(z) = 1/(1 + e^{-z})$ — smooth, saturating
- **ReLU:** $\sigma(z) = \max(0, z)$ — non-saturating, enables deep training
- **GELU:** $\sigma(z) = z \cdot \Phi(z)$ — smooth ReLU variant, used in Transformers

Parameters: $W^{(l)} \in \mathbb{R}^{k_l \times k_{l-1}}$, $b^{(l)} \in \mathbb{R}^{k_l}$. Total: $\sum_l k_l(k_{l-1} + 1)$.

Convolutional Neural Networks (CNN)

Core idea: Exploit local spatial structure by sharing weights across spatial locations.

Convolution layer: A filter $K \in \mathbb{R}^{h \times w}$ slides over the input, computing local dot products. Each filter detects a single feature (edge, texture, or pattern).

Properties:

- **Translation equivariance:** Shifting the input shifts the feature map identically
- **Locality:** Each output pixel depends only on a small receptive field
- **Parameter sharing:** Same filter applied everywhere, fewer parameters than MLP

Pooling: Downsample feature maps (max pooling, average pooling) $\rightarrow$ reduces spatial resolution, builds invariance to small translations.

Recurrent Neural Networks (RNN)

Core idea: Process sequences by maintaining a hidden state that carries information across time steps.

Vanilla RNN:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$

The hidden state h_t summarizes all past inputs $\{x_1, \dots, x_t\}$.

Problem — vanishing gradients: Backpropagation through time multiplies the weight matrix W_h at each step. If $\|W_h\| < 1$, gradients decay exponentially — long-range dependencies become impossible to learn.

LSTM / GRU: Add gating mechanisms (input gate, forget gate, output gate) that allow the network to selectively remember or forget information, mitigating vanishing gradients.

The Transformer Architecture

Core idea (Vaswani et al., 2017): Replace recurrence with **self-attention** — every token attends to every other token directly.

Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Q, K, V are learned projections of the input. The $\sqrt{d_k}$ scaling prevents softmax saturation.

Multi-head attention: Run h attention heads in parallel, concatenate outputs — allows attending to different representation subspaces.

Position encoding: Since attention is permutation-invariant, add sinusoidal or learned position embeddings.

Architecture: Input $\rightarrow$ Self-Attention + FFN $\rightarrow \dots \times L$ layers $\rightarrow$ Output

Key advantage: $O(1)$ sequential operations between any two positions — fully parallelizable.

Residual Connections (ResNet)

Problem: Very deep networks ($L > 50$) suffer from vanishing gradients even with ReLU. The signal degrades through many layers.

Residual block (He et al., 2015):

$$h^{(\ell+1)} = h^{(\ell)} + \mathcal{F}(h^{(\ell)}; W^{(\ell)})$$

Instead of learning $h^{(\ell+1)}$ from scratch, learn the **residual** $\mathcal{F}$ — the *correction* to the identity mapping.

Why this helps: The gradient passes through identity: $\frac{\partial h^{(\ell+1)}}{\partial h^{(\ell)}} = I + \frac{\partial \mathcal{F}}{\partial h^{(\ell)}}$. The I term prevents vanishing — even if $\mathcal{F}$ has small derivatives, gradients flow.

Impact: ResNets enabled 100+ layer networks. Skip connections are now standard in almost all modern architectures (Transformers, UNets, etc.).

Backpropagation: The Computational Engine

Goal: Compute $\partial\mathcal{L}/\partial\theta$ for all parameters θ in a computational graph.

Chain rule, applied recursively:

$$\frac{\partial \mathbf{h}^{(L)}}{\partial \theta^{(l)}} = \prod_{k=l+1}^L \left(\sigma' \cdot W^{(k)} \right) \cdot \frac{\partial \mathbf{h}^{(l)}}{\partial \theta^{(l)}}$$

Forward pass: Compute activations $h^{(1)}, h^{(2)}, \dots, h^{(L)}$, store intermediate values.

Backward pass: Compute gradients from output to input, reusing stored activations. Each layer's gradient depends only on the next layer's gradient (local computation).

Cost: One forward + one backward pass = $\sim 2 \times$ the cost of inference. **Memory:** must store all intermediate activations ($O(\# \text{ layers} \times \text{batch size})$).

Vanishing gradient problem: For deep networks ($L \gg 1$), the product $\prod_k \sigma' \cdot W^{(k)}$ can decay to zero (sigmoid) or explode — the key motivation for ReLU and residual connections.

Loss Functions: Regression

Regression (continuous labels): Predict a real value.

$$\text{MSE: } \mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2, \quad \text{MAE: } \mathcal{L} = \frac{1}{n} \sum_{i=1}^n |y_i - f(x_i)|$$

MSE (L^2) heavily penalizes large errors — sensitive to outliers. MAE (L^1) is more robust.

Bayesian interpretation: MSE = negative log-likelihood under Gaussian noise model $y = f(x) + \varepsilon, \varepsilon \sim \mathcal{N}(0, \sigma^2)$. MAE corresponds to Laplace noise.

Both assume the *scale* of the output matters additively. For classification, that's the wrong assumption.

Loss Functions: Classification

Classification (discrete labels): Final layer = **softmax**:

$$\text{Softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Converts raw logits z into a probability distribution over K classes.

Cross-entropy loss (negative log-likelihood):

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log \hat{p}_{ik}$$

y_{ik} = one-hot true label, $\hat{p}_{ik}$ = predicted probability. Minimizing cross-entropy = maximum likelihood under a categorical model.

Exercise 7

For binary classification, show that the MSE gradient vanishes when the prediction is confident *and wrong* (sigmoid output $\rightarrow 0$ when $y = 1$), whereas the cross-entropy gradient does not.

Regularization: L_2 and Dropout

Regularization aims to reduce overfitting.

L_2 **regularization** (weight decay):

$$\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Dropout (Srivastava et al., 2014): During training, randomly set neurons to zero with probability p . Forces redundancy — the network cannot rely on any single neuron. At test time, all neurons are active (scaled by $1 - p$).

Regularization: Early Stopping & BatchNorm

Early stopping: Monitor validation loss. Stop when it starts rising (L-curve). The optimizer starts near the initialization point and moves toward more complex solutions; stopping early keeps the solution simple. This is the **implicit bias** of SGD: early iterations fit low-frequency (simple) patterns first.

Batch Normalization (Ioffe & Szegedy, 2015): Applied **before** the activation:

Linear $\rightarrow$ BN $\rightarrow$ ReLU. Normalizes pre-activations to zero mean, unit variance:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

γ, β learnable scale/shift. BN after activation also works in practice, but the original placement is before.

Essential Tool in ODE

Grönwall's Inequality

Many training proofs rely on continuous-time (gradient-flow) models. The key tool:

Theorem (Grönwall)

Let $y(t) \geq 0$ on $[a, b]$ with $y'(t) \leq \alpha(t)y(t)$. Then:

$$y(t) \leq y(a) \exp\left(\int_a^t \alpha(s) ds\right)$$

Grönwall converts differential inequalities into explicit bounds; it is the bridge from local (derivative) information to global (function value) guarantees.

The Non-Convex Landscape

Why Neural Network Loss Is Non-Convex

Consider a simple 2-layer network even without activation

$$f(x; w_1, w_2) = w_2^\top w_1 x$$

where $w_2 \in \mathbb{R}^{n \times 1}$, $w_1 \in \mathbb{R}^{n \times 1}$ and $x \in \mathbb{R}$.

The composition of parameters makes the loss non-convex (in w_1, w_2):

$$\mathcal{L}(w_1, w_2) = \frac{1}{2}(w_2^\top w_1 x - y)^2$$

Symmetry: Permuting neurons or flipping signs produces identical functions:

$f(x; w_1, w_2) = f(x; -w_1, -w_2)$ for odd σ like $\tanh$.

Local minimum: $w_2^\top w_1 = \frac{y}{x}$ contains a hyperbolic manifold (non-convex).

Critical Points: Minima, Maxima, and Saddles

Definition (Critical Points of $f : \mathbb{R}^n \rightarrow \mathbb{R}$)

A point $\mathbf{x}^*$ is **critical** if $\nabla f(\mathbf{x}^*) = 0$. Classified by the Hessian $H = \nabla^2 f(\mathbf{x}^*)$:

- **Local minimum:** All eigenvalues of H are ≥ 0
- **Local maximum:** All eigenvalues of H are ≤ 0
- **Saddle point:** Some eigenvalues > 0 , some < 0

The **index** of a saddle is the number of negative eigenvalues — the number of descent directions.

Low-dimensional intuition: In 2D, saddles look like a mountain pass. In n dimensions, a saddle point has k ascent directions and $n - k$ descent directions.

Saddle Points Dominate in High Dimensions

The probability argument: A critical point is a minimum if *all* n eigenvalues of the Hessian are positive. For a random symmetric matrix (e.g., from a random loss surface):

$$\mathbb{P}[\text{all eigenvalues} > 0] \approx \left(\frac{1}{2}\right)^n$$

This is **exponentially small** in the dimension n . Conversely:

$$\mathbb{P}[\text{at least one eigenvalue} < 0] \approx 1 - 2^{-n} \rightarrow 1$$

In practice, $n \sim 10^6$ – 10^9 . The probability of encountering a genuine local minimum by chance is effectively zero. **Every critical point is overwhelmingly likely to be a saddle.**

Implication: Gradient descent doesn't get stuck in saddle points; it gets slowed down by saddle points, where the gradient vanishes but descent directions exist.

Morse Theory: Classifying the Critical Points

Theorem (Morse Lemma)

Near a **non-degenerate** critical point p of index k , there exist local coordinates such that:

$$f(x) = f(p) - x_1^2 - \cdots - x_k^2 + x_{k+1}^2 + \cdots + x_n^2$$

What this means: Locally, every critical point looks like a quadratic form. The index k counts negative directions (descent), $n - k$ counts positive directions (ascent).

- **Index 0:** All positive — local minimum
- **Index k , $0 < k < n$:** Saddle point — k ways to descend, $n - k$ ways to ascend
- **Index n :** All negative — local maximum

Analog: Spin Glass Model

Large neural networks (MLP) at initialization behave like **spin glasses** in statistical physics, with random, highly non-convex energy landscapes. (Choromanska et al. (2015))

p -spin spherical spin glass:

$$\mathcal{H}(\sigma) = -\frac{1}{N^{(p-1)/2}} \sum_{i_1 < \dots < i_p} J_{i_1 \dots i_p} \sigma_{i_1} \cdots \sigma_{i_p},$$

- Spins $\sigma \in \mathbb{R}^N$ are continuous, constrained to $\|\sigma\| = \sqrt{N}$ (spherical).
- Couplings $J_{i_1 \dots i_p} \sim \mathcal{N}(0, 1)$, Gaussian disorder.
- $p = 3$: simplest non-trivial case.
- **Ground State** means the minimum of $\mathcal{H}/N$.

Theoretical Results (Random Matrix Theory)

When $N \rightarrow \infty$, the global minimum $E_0 \rightarrow -1.658$. (Auffinger et al. (2013)). Let $E_c = -2\sqrt{p/(p-1)} = 1.633$.

- Above E_c : almost all critical points have a large index (saddle points).
- Between E_0 and E_c : local minima dominate.
- Below E_0 : impossible

At a finite N , the global minimum will be above E_0 , and the gap vanishes as $N \rightarrow \infty$.

Demo for $N = 3$

There are 3 distinct local minima for $N = 3$. The global minimum (which depends on the sampled J) is approximately -1.535 .

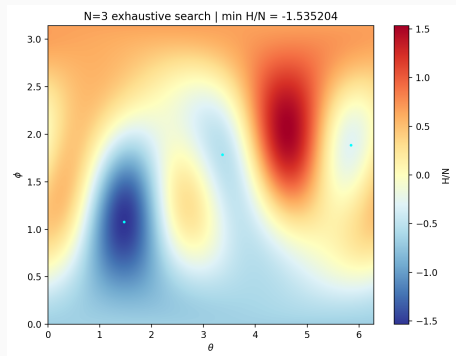


Figure 2: Grid Searched $\mathcal{H}/N$ for $N = 3$ in spherical coordiante.

Exercise 8

Use whatever method to find the minimum of $\mathcal{H}/N$ for $N = 50$, and confirm the point is truly a local minimum by checking the Hessian's eigenvalues.

Run your method for different initializations. What does the distribution of the final minima look like?

Essential References

1. Shannon (1948). *A Mathematical Theory of Communication*.
2. Chernoff (1952). *A Measure of Asymptotic Efficiency for Tests of a Hypothesis*.
3. Hoeffding (1963). *Probability Inequalities for Sums of Bounded Random Variables*.
4. Rosenblatt (1958). *The Perceptron: A Probabilistic Model for Information Storage*.
5. Rumelhart, Hinton & Williams (1986). *Learning Representations by Back-Propagating Errors*.
6. Vaswani et al. (2017). *Attention Is All You Need*.
7. Choromanska et al. (2015). *The Loss Surfaces of Multilayer Networks*.
8. Dauphin et al. (2014). *Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization*.
9. Auffinger et al. (2013). *Random matrices and complexity of spin glasses*.